AUTONEX

Ficha técnica del proyecto

Marketplace de autopartes y repuestos de motos · Versión demo · gnuino.com.ar/AUTONEX

En una frase: es un marketplace de autopartes hecho con **React y Vite** en el frontend, una **API REST en Node.js con Fastify**, base de datos **PostgreSQL**, pagos con **Mercado Pago Checkout Pro** y publicado en un servidor **Linux con nginx y systemd**. Todo el código está escrito en **JavaScript**.

1 Arquitectura



El frontend nunca habla directo con la base, ni con Mercado Pago ni con el servicio de mails: todo pasa por la API.

2 Tecnologías por capa

Frontend carpeta client/

- **React 18**: interfaz armada por componentes.
- **Vite 6**: compila el proyecto y levanta el entorno de desarrollo.
- **React Router 7**: navegación entre páginas sin recargar.
- **CSS propio**, sin frameworks; responsive (computadora y celular).
- Íconos **Lucide**; tipografías **Barlow** y **JetBrains Mono**.

Backend carpeta server/

- **Node.js 20** con **Fastify 5**: API REST en `/api/…`.
- Validación de cada pedido a la API con esquemas JSON.
- **JWT** para las sesiones y **bcrypt** para guardar las contraseñas encriptadas.
- Roles: **comprador**, **vendedor** y **administrador**.
- Importación de datos desde **CSV, Excel y XML** (csv-parse, exceljs, fast-xml-parser).
- **Mails por SMTP (Gmail, con `nodemailer`) o Resend**: confirmación de la cuenta, comprobante, recuperación de contraseña y reintegros.

Base de datos

- **PostgreSQL 17**, conectada con la librería **pg**.
- **Migraciones SQL numeradas** (`001` a `013`): la estructura cambia sin perder datos.
- Las compras se guardan en **transacciones**: precios, stock y envíos se recalculan en el servidor.

Pagos

- **Mercado Pago Checkout Pro** con su SDK oficial para Node.
- El backend crea la orden de pago y, al volver, **consulta el pago a Mercado Pago** (no confía en la URL).
- **Webhook**: Mercado Pago avisa al servidor cuando un pago se aprueba.

Servidor y publicación

- Servidor **Linux Debian 13** (gnuino.com.ar).
- **nginx** como proxy con **HTTPS**.
- **systemd**: la app corre como servicio y arranca sola si el servidor se reinicia.

Pruebas

- **65 tests automáticos** con el test runner de Node (`node:test`).
- Prueban la API completa sobre una **base de datos de prueba separada** y un Mercado Pago simulado.

3 Qué hace el sistema

- Búsqueda de repuestos por texto, código de pieza o **vehículo** (auto o moto: marca, modelo, año, versión).
- Compatibilidades de cada repuesto con los vehículos.
- Varios vendedores por producto: se comparan precios, stock y ubicación.
- Carrito con productos de distintos vendedores en una sola compra.
- Checkout **con o sin cuenta**: entrega por retiro en el local o envío a domicilio.
- Pago con Mercado Pago y pantalla de resultado (aprobado, pendiente o rechazado).
- **Seguí tu compra**: estado de cada pedido y número de seguimiento del envío.
- **Comprobante** para descargar en PDF, compartir y también por mail.
- Cuentas con **confirmación de email**, recuperación de contraseña y opción de **eliminar la cuenta**.
- **Términos, privacidad y botón de arrepentimiento** (Ley 24.240).
- **Panel del vendedor**: logo del local, publicaciones propias con fotos y descripción, precios, stock y pedidos. Las fotos se validan y optimizan con `sharp` (WebP) y se guardan en PostgreSQL.
- **Panel de administración**: catálogo e importaciones de datos.

4 De dónde salen los datos

Dato	Fuente
Marcas y modelos de autos	Datos abiertos de la **DNRPA** (registro automotor argentino)
Modelos de motos, fotos y logos	**Wikidata** y **Wikimedia Commons**, con créditos en la página "Créditos"
Repuestos, precios y vendedores	Datos de **demostración** (el proyecto es una demo)

5 Estructura del proyecto

Carpeta	Contenido
`client/src/pages`	Pantallas: inicio, catálogo, producto, carrito, checkout, compras, paneles
`client/src/components`	Piezas reutilizables: encabezado, selector de vehículo, tarjetas de producto
`server/src/routes`	Rutas de la API (catálogo, compras, pagos, panel del vendedor, administración)
`server/src/services`	Lógica de negocio: compras, envíos, pagos, Mercado Pago
`server/db`	Migraciones SQL y carga inicial de datos (seed)
`server/test`	Tests automáticos de la API
`deploy`	Configuración de nginx y del servicio de systemd